

GUÍA DE CLASE

De una idea a una web publicada con inteligencia artificial

Método práctico con Claude Code o Codex, Next.js o Astro, GitHub y Vercel

01 Decidir	02 Construir	03 Verificar	04 Publicar
----------------------	------------------------	------------------------	-----------------------

Clase en vivo · Domingo 23 de agosto de 2026 · CrececonIA

Versión de trabajo para construir, revisar e iterar una presencia web moderna y segura.

Cómo usar esta guía

Esta guía acompaña la presentación de la clase y funciona después como manual operativo. No exige saber programar, pero sí tomar decisiones claras, revisar cambios y tratar la publicación como un proceso controlado.

RESULTADO MÍNIMO Una carpeta con contexto, una V1 que se puede abrir, un repositorio en GitHub y una URL de Preview o Producción en Vercel.

Ruta de 150 minutos

Hora	Bloque	Evidencia
18:00	Alcance y resultado	Tipo de proyecto elegido
18:10	Prompts y contexto	Instrucción estructurada
18:25	Herramienta y carpeta	Proyecto abierto con archivos base
18:40	Referencias, arte y stack	Brief + mapa de información
19:00	Construcción guiada	Primera visualización
19:30	Revisión de V1	Problema prioritario definido
19:40	GitHub y Vercel	Repositorio + deployment
20:00	Iteraciones y módulos	Rama con una mejora
20:15	QA y optimización	Checklist técnico
20:25	Cierre	Siguiente paso de siete días

1. Elegir un proyecto que puedas terminar

La primera decisión no es el framework. Es qué debe lograr la página y qué acción debe realizar la persona. Un alcance pequeño permite aprender el ciclo completo; uno excesivo deja una colección de pantallas sin verificar.

Tipo	Úsalo cuando	CTA habitual
Landing	Una oferta, campaña o servicio necesita conversión directa.	Agendar, cotizar, comprar, registrarse
Corporativo	La empresa necesita explicar capacidades, sectores, casos y contacto.	Solicitar reunión o propuesta
Portafolio	La prueba principal es el trabajo realizado.	Ver proyectos o contactar
Contenido	La adquisición depende de artículos, recursos o conocimiento.	Suscribirse o explorar
E-commerce	Hay catálogo, precio, pago, pedido y operación.	Agregar al carrito o comprar

REGLA Si la versión inicial necesita login, pagos, inventario y un panel administrativo, ya no es una landing. Divide el proyecto en fases.

Definición en una frase

Plantilla

Construiremos una [landing/web corporativa/portafolio] para [audiencia] que necesita [problema o deseo], con el objetivo de que [acción principal].

2. Prompts básicos que producen trabajo verificable

Un prompt de proyecto es una especificación con puntos de control. La fórmula siguiente sirve para construir componentes, corregir errores, hacer refactors y planificar funciones complejas.

Parte	Pregunta	Ejemplo
Contexto	¿Dónde estamos?	Landing Next.js; brief listo; aún sin componentes.
Objetivo	¿Qué resultado único buscamos?	Crear la primera versión del hero.
Especificaciones	¿Qué debe incluir?	Titular, bajada, CTA, prueba y navegación.
Restricciones	¿Qué no debe ocurrir?	No inventar testimonios ni agregar librerías.
Verificación	¿Cómo se comprueba?	390 px, teclado, CTA, build y consola.
Formato	¿Cómo entrega el trabajo?	Plan, archivos tocados, pruebas y pendientes.

Prompt básico reutilizable

```

CONTEXTO: [proyecto y estado actual]
OBJETIVO: [resultado en una frase]
ESPECIFICACIONES: [requisitos concretos]
RESTRICCIONES: [scope y decisiones cerradas]
VERIFICACIÓN: [pruebas observables]
FORMATO: [plan, archivos y resumen]

```

BUEN HÁBITO Una tarea por turno. Para un cambio amplio, pide plan y espera aprobación antes de editar.

3. El contexto: memoria externa del proyecto

El agente no conoce tu empresa ni tus decisiones. El contexto útil vive en archivos pequeños, actualizados y cercanos al código. Así una nueva sesión puede retomar el proyecto sin depender de recordar una conversación antigua.

Archivo	Contenido mínimo
README.md	Cómo instalar, ejecutar, probar y publicar.
BRIEF.md	Audiencia, problema, oferta, CTA, tono, contenido y límites.

Archivo	Contenido mínimo
DECISIONES.md	Acuerdos cerrados, motivo, fecha e impacto.
TAREAS.md	Próximos resultados observables, no ideas vagas.
REFERENCIAS.md	URL o captura, patrón útil, adaptación y lo que no se copiará.

Primer prompt al abrir la carpeta

Lee README.md, BRIEF.md, DECISIONES.md, TAREAS.md y REFERENCIAS.md. Resume qué entiendes que construiremos, qué información falta y qué riesgos ves. No modifiques archivos todavía.

Carpeta y permisos

1. Crea una carpeta exclusiva para el proyecto; evita abrir Escritorio, Descargas o Documentos completos.
2. Abre esa carpeta en Claude Code o Codex y confirma la ruta visible.
3. Empieza autorizando lectura. Revisa instalaciones, comandos de red y cambios masivos.
4. Guarda secretos en .env.local y en el panel de Vercel; nunca en archivos versionados.
5. Antes de cada entrega, pide un resumen de archivos modificados y pruebas ejecutadas.

4. Referencias y dirección de arte

Una referencia sirve para señalar una decisión: jerarquía, navegación, ritmo, tratamiento de imágenes, tipo de prueba social o interacción. No autoriza copiar marca, texto, fotografías ni código con licencia incompatible.

Fuentes útiles

Fuente	Para qué	Cuidado
Mobbin	Flujos y patrones de productos reales.	No todo patrón sirve para una landing.
Awwwards	Dirección visual y narrativa.	Filtra performance y accesibilidad.

Fuente	Para qué	Cuidado
Land-book / Lapa Ninja	Landings por rubro y estructura.	Evita clonar el conjunto completo.
21st.dev / shadcn/ui	Componentes y secciones editables.	Revisa licencia, dependencias y semántica.
Sitios competidores	Lenguaje, oferta y expectativas de mercado.	Busca diferenciación, no imitación.

Ficha de referencia

REFERENCIAS.md

```
URL o archivo: [referencia]
Patrón útil: [qué observamos]
Decisión propia: [cómo lo adaptaremos]
No copiar: [marca, texto, asset o interacción]
Prueba: [cómo verificaremos que funciona]
```

Cinco decisiones visuales

- Dirección concreta: editorial, Swiss, industrial, cálida orgánica, tecnológica sobria, entre otras.
- Paleta semántica: acción principal, fondos, texto, borde, éxito, advertencia y error.
- Tipografía: máximo dos familias y una escala con contraste real.
- Jerarquía y ritmo: no todas las secciones deben tener el mismo peso ni el mismo espacio.
- Reglas que no se rompen: por ejemplo, sin gradientes decorativos, sin sombras genéricas y sin testimonios inventados.

5. Elegir Astro o Next.js

Ambos son stacks modernos. La decisión depende del comportamiento dominante del proyecto, no de cuál aparece más en redes sociales.

Criterio	Astro	Next.js
Contenido	Excelente para sitios corporativos, editoriales y marketing.	Excelente si el contenido convive con aplicación.

Criterio	Astro	Next.js
JavaScript	Envía poco por defecto y agrega islas interactivas.	React y renderizado híbrido con más capacidades de aplicación.
Servidor	Estático por defecto; adaptadores para render bajo demanda.	Rutas, acciones y lógica de servidor integradas.
Complejidad	Muy buena base para páginas principalmente informativas.	Conviene cuando habrá sesión, panel, datos o flujos complejos.
Vercel	Deploy estático sin configuración extra; adaptador si es dinámico.	Integración directa y previews por Git.

DECISIÓN RÁPIDA Si el 80% es contenido público, evalúa Astro. Si habrá autenticación, datos, lógica de servidor o crecimiento hacia aplicación, usa Next.js. Verifica siempre la documentación de la versión instalada.

Prompt de setup moderno

Crea el proyecto con la versión estable actual de [Next.js/Astro], TypeScript strict y el sistema de estilos acordado. Usa el scaffold oficial. Antes de generar componentes, muestra versiones, árbol de carpetas y comandos. Ejecuta el build y no ignores warnings.

6. Prompt maestro para pedir el sitio completo

Copia este prompt en un archivo PROMPT-INICIAL.md y reemplaza cada campo entre corchetes. Aunque pide el sitio completo, exige un plan y una implementación por etapas para evitar un cambio monolítico.

PROMPT MAESTRO

CONTEXTO
Lee README.md, BRIEF.md, DECISIONES.md y REFERENCIAS.md. El proyecto es una [landing / web corporativa / portafolio] para [tipo de negocio] en [mercado o ubicación]. La audiencia principal es [audiencia] y llega desde [canal].

OBJETIVO
Construir una primera versión responsive que explique [oferta], demuestre [prueba] y consiga que la persona [CTA principal].

STACK

Usa [Next.js con App Router / Astro] en su versión estable actual, TypeScript strict y [Tailwind CSS / CSS del proyecto]. No uses HTML suelto como arquitectura final. Lee la documentación instalada antes de asumir APIs.

ARQUITECTURA DE INFORMACIÓN

Incluye, en este orden: [navegación], [hero], [problema/beneficios], [oferta o servicios], [proceso], [prueba social real], [FAQ], [CTA final], [footer]. Agrega sólo páginas secundarias justificadas por el mapa de información.

CONTENIDO

Usa los textos, datos y archivos proporcionados. No inventes clientes, métricas, precios, testimonios, certificaciones ni integrantes. Si falta algo, usa un marcador visible [PENDIENTE: dato requerido].

DIRECCIÓN VISUAL

Dirección: [ej. editorial contemporánea]. Paleta: [hex y función]. Tipografía: [familias]. Referencias: [URLs o capturas] y el patrón específico que tomaremos de cada una. Mantén una identidad propia y evita el hero genérico con gradiente y tres cards idénticas.

EXPERIENCIA

Diseña primero para 390 px y adapta a tablet y escritorio. Usa HTML semántico, navegación por teclado, foco visible, contraste legible, botones táctiles y prefers-reduced-motion. Optimiza imágenes y limita JavaScript y animaciones.

FUNCIONES

La CTA debe [abrir WhatsApp / agendar / enviar formulario]. Si hay formulario, incluye estados de envío, éxito y error, validación cliente/servidor y protección anti-spam. No expongas secretos en el navegador.

SEO Y DESCUBRIMIENTO

Incluye title y description únicos, canonical, Open Graph, favicon, sitemap, robots, URLs descriptivas y JSON-LD correcto para [Organization / LocalBusiness / Service / Product]. Mantén contenido principal en texto visible. llms.txt es opcional y debe tratarse como complemento experimental.

PROCESO

1. No edites todavía. Resume el brief, enumera faltantes y propone arquitectura, archivos y plan por etapas.
2. Espera aprobación.
3. Implementa estructura y contenido antes de detalle visual.
4. Abre o captura la primera visualización y revisa 390 px y escritorio.
5. Ejecuta tests, typecheck y build; corrige errores sin ocultarlos.
6. Resume archivos modificados, dependencias, pruebas, riesgos y pendientes.

RESTRICCIONES

No cambies el stack, no agregues CMS, analítica, librerías o integraciones sin explicar necesidad, costo y alternativa. No toques otras carpetas. No despliegues a producción sin aprobación.

CRITERIOS DE ACEPTACIÓN

La oferta se entiende en cinco segundos; CTA visible y funcional; sin overflow horizontal;

teclado y foco funcionan; imágenes tienen dimensiones y alt cuando corresponde; no quedan datos falsos; build limpio; consola sin errores relevantes; existe una lista de QA pendiente para producción.

7. Generar y revisar la primera visualización

6. Pide el plan y confirma que las secciones responden el brief.
7. Implementa estructura, contenido y CTA. Deja animaciones y detalles para otra iteración.
8. Levanta el entorno de desarrollo y abre la página en 390 px.
9. Comprueba qué es, para quién, por qué creer y qué hacer ahora.
10. Haz clic en navegación, CTA, enlaces y formulario. Revisa estados de error.
11. Guarda una captura y formula un solo problema observable para la V2.

Prompt para revisar la V1

Revisa la página como visitante nuevo. Evalúa claridad de oferta, jerarquía, confianza, CTA, móvil, accesibilidad y errores funcionales. No edites. Entrega hallazgos priorizados como: problema observable, impacto, evidencia y cambio mínimo recomendado.

Evita	Usa
Hazlo más bonito.	El CTA queda bajo el pliegue en 390 px; reduce texto sin perder la promesa.
Se ve raro.	La jerarquía da el mismo peso a titular, navegación y cards; define un foco.
Cambia todo.	Ajusta sólo el hero y conserva navegación, tokens y demás secciones.
Arregla como sea.	Diagnostica causa raíz, propone reparación mínima y espera aprobación.

8. Crear GitHub y trabajar de forma segura

12. Crea la cuenta en github.com/signup, verifica el correo y activa autenticación de dos factores.
13. Crea un repositorio con nombre claro. Para trabajo de cliente, comienza privado salvo acuerdo distinto.
14. Confirma `.gitignore` antes del primer commit. Debe excluir `.env*`, dependencias, builds y archivos privados.

15. Revisa git status. Sólo después agrega archivos, crea el commit y hace push.
16. Usa main como versión estable y una rama feature/... por iteración.

Primer push

```
git status
git add .
git commit -m "feat: primera versión de la landing"
git branch -M main
git remote add origin [URL_DEL_REPOSITORIO]
git push -u origin main
```

Ramas, pull requests y merge

Elemento	Función	Regla práctica
Rama	Aísla una mejora.	feature/hero, fix/formulario, perf/imagenes
Commit	Fotografía un cambio coherente.	Pequeño, descriptivo y verificable
Pull request	Muestra diff, checks y Preview.	Explica qué, por qué, cómo probar y riesgos
Merge	Integra a main.	Sólo con revisión y checks verdes
Revert	Crea un cambio que deshace otro.	Preferible a reescribir historia compartida

PROTECCIÓN RECOMENDADA En proyectos de equipo, exige pull request y status checks antes de mergear a main; bloquea force push y eliminación de la rama protegida.

9. Crear Vercel y desplegar desde GitHub

17. Crea cuenta en vercel.com e inicia sesión con GitHub.
18. Autoriza sólo la organización y repositorios necesarios.
19. Selecciona Add New → Project e importa el repositorio correcto.
20. Revisa framework preset, root directory, comando de build y output.
21. Agrega variables de entorno en Development, Preview o Production según corresponda.
22. Despliega, abre la URL y revisa logs si falla.

Flujo seguro de despliegue

Entorno	Se crea con	Uso
Local	Servidor de desarrollo	Construcción rápida
Preview	Push a rama o pull request	QA y aprobación sin afectar producción
Production	Merge o push a la rama de producción	Dominio público

IMPORTANTE Una variable guardada en Vercel debe asignarse al entorno correcto. Los valores de Preview pueden ser distintos de Producción. Nunca conviertas una clave secreta en variable pública del navegador.

10. V1, V2, V3 y V4: iterar sin perder control

Versión	Pregunta	Salida
V1 funcional	¿Se entiende y se puede completar la acción?	Estructura, contenido, CTA y build
V2 visual	¿La jerarquía guía la lectura?	Tokens, espaciado, tipografía e imágenes
V3 confianza	¿Existe evidencia y baja fricción?	Prueba, FAQ, formularios y mensajes
V4 producción	¿Resiste tráfico, bots y errores?	QA, performance, SEO, seguridad y monitoreo

Prompt de iteración

En la captura adjunta, el problema observable es [problema]. Ajusta sólo [sección] para conseguir [resultado medible]. Conserva [elementos que funcionan]. Revisa 390 px y escritorio, ejecuta build y muestra el diff antes de cerrar.

Archivos propios e imágenes

- Usa nombres descriptivos: equipo-sergio-reunion.webp, no IMG_4837-final2.png.

- Guarda assets públicos en la carpeta que el framework define para estáticos; los privados nunca van allí.
- Genera tamaños adecuados y formatos modernos; conserva originales fuera del bundle cuando corresponda.
- Define width y height para evitar saltos de layout.
- Escribe alt que explique el contenido o propósito; usa alt vacío si la imagen es puramente decorativa.

Módulos externos

Los módulos aceleran patrones repetidos, pero también introducen dependencias, estilos, JavaScript y decisiones ajenas. Evalúa cinco cosas: licencia, mantenimiento, accesibilidad, peso y compatibilidad con tu sistema visual.

Prompt para integrar un módulo

Integra [módulo] únicamente en [sección]. Adáptalo a tokens, tipografía y componentes existentes. Mantén semántica, teclado y reduced motion. No dupliques dependencias. Ejecuta build, revisa bundle y compara visualmente antes/después.

11. Pruebas y QA

Una página puede compilar y aun así estar rota para el usuario. La salida requiere pruebas automáticas, revisión de navegador y validación humana.

Capa	Qué prueba	Herramienta
Build y tipos	Errores de compilación y contratos.	npm run build / typecheck
Unitario	Funciones y componentes aislados.	Vitest / Jest
Integración	Formulario, API, validación y estados.	Vitest + mocks controlados
E2E	Recorrido real en navegador.	Playwright / Cypress
Visual	Overflow, jerarquía y responsive.	Capturas y comparación
Humano	Claridad, confianza y fricción.	Prueba con personas

Checklist antes de compartir

- La propuesta se entiende en cinco segundos y el CTA principal es visible.
- No hay overflow horizontal en 390 px ni componentes cortados en tablet.
- Teclado, foco, labels, mensajes de error y contraste son utilizables.
- Todos los enlaces, WhatsApp, agenda, mail y formularios se probaron.
- No quedan lorem ipsum, datos inventados, URLs de desarrollo ni console.log sensibles.
- Build y tests pasan; la consola y red no muestran errores relevantes.
- Existe una Preview identificable y un commit asociado.

Prompt de QA

```
Prueba la web como visitante nuevo en 390x844, 768x1024 y 1440x900. Revisa navegación, teclado, CTA, enlaces, formulario, mensajes, overflow, consola, red y 404. No edites. Entrega hallazgos con severidad, pasos, esperado, real y evidencia.
```

12. Performance, accesibilidad y seguridad

Performance

- Comprime imágenes, usa tamaños responsivos y evita descargar el original gigante en móvil.
- Limita familias y pesos tipográficos; usa carga del framework cuando exista.
- Mantén JavaScript fuera de secciones que sólo muestran contenido.
- Revisa dependencias grandes y animaciones que bloquean el hilo principal.
- Usa Lighthouse como señal de laboratorio y combina con métricas reales cuando haya tráfico.

Accesibilidad

- Una jerarquía de headings coherente y landmarks semánticos.
- Contraste suficiente, foco visible y áreas táctiles cómodas.
- Labels asociados, errores explicativos y estados anunciados.
- Navegación completa con teclado y soporte prefers-reduced-motion.
- Alt útil sólo cuando la imagen aporta información.

Seguridad

- Mínimo privilegio en GitHub, Vercel, conectores y tokens.
- Variables de entorno en servidor; nada secreto con prefijo público.
- Validación de entrada en servidor, límites de frecuencia y anti-spam.
- Dependencias revisadas y actualizadas; evita copiar scripts desconocidos.

- Backups, commits y procedimiento de rollback probado.

13. SEO técnico y contenido

SEO comienza con una página útil y rastreable. Los metadatos no compensan una oferta confusa ni contenido genérico.

Área	Implementación mínima
Metadata	Title, description, canonical, Open Graph y Twitter image.
Indexación	robots, sitemap, status correctos y noindex donde corresponda.
Arquitectura	URLs descriptivas, navegación interna y breadcrumbs si aporta.
Contenido	Respuesta directa, lenguaje de audiencia, autoría, fechas y evidencia.
Schema	JSON-LD específico y fiel al contenido visible.
Performance	Carga rápida, estabilidad visual y experiencia móvil.

Prompt SEO

```
Audita esta web para SEO técnico. Revisa metadata, canonical, Open Graph, sitemap, robots, status, URLs, headings, enlaces internos, imágenes, contenido duplicado y JSON-LD. Separa errores, oportunidades y elementos correctos. No inventes schema ni datos que la página no muestra.
```

14. GEO, LLM, robots y contenido citable

GEO suele describir optimización para motores generativos. No existe una receta que garantice ser citado. La estrategia sensata es producir contenido claro, verificable, accesible y estructurado que una persona y una máquina puedan interpretar de la misma forma.

- Responde preguntas concretas al inicio de cada sección.
- Incluye nombres, ubicaciones, servicios, condiciones y fechas explícitas.
- Diferencia hechos, opinión, promesa y ejemplo.
- Cita fuentes primarias y muestra autoría y actualización.
- Usa HTML semántico y datos estructurados fieles al texto visible.

- Evita contenido inflado creado sólo para repetir palabras clave.

Archivo/regla	Qué hace	Qué no hace
robots.txt	Administra qué rutas pueden rastrear bots obedientes.	No protege información privada.
noindex	Pide no incluir una página en resultados.	No impide que alguien con URL acceda.
sitemap.xml	Enumera URLs importantes para descubrimiento.	No garantiza indexación ni ranking.
llms.txt	Puede resumir recursos para algunos consumidores.	No es garantía universal ni control de seguridad.

SEGURIDAD Si un contenido debe ser privado, usa autenticación y autorización reales. Nunca confíes en robots.txt para ocultar datos.

15. Formularios, correo e integraciones

Un formulario de producción es un flujo, no sólo campos. Debe recibir, validar, proteger, entregar y confirmar.

- Define los campos mínimos y una base legal/consentimiento cuando corresponda.
- Valida en cliente para ayudar y en servidor para confiar.
- Aplica anti-spam: honeypot, CAPTCHA selectivo o rate limit según riesgo.
- Envía correo con Resend, Formspree o una función serverless; la clave queda en servidor.
- Devuelve estados de cargando, éxito, error y reintento.
- Registra errores sin almacenar más datos personales de los necesarios.

Prompt para formulario

```
Implementa el formulario [campos] con validación cliente y servidor, estados loading/success/error, honeypot y rate limit. Envía mediante [Resend/Formspree] desde servidor. Guarda la clave en variables de entorno. Agrega política de privacidad y prueba envío exitoso, dato inválido, duplicado, spam y caída del proveedor.
```

Funciones frecuentes para una web corporativa

Necesidad	Opción inicial	Cuando crecer
Contacto	Formulario + correo	CRM y asignación automática
Agenda	Calendly/Cal.com	Agenda propia con disponibilidad
Newsletter	Kit/Brevo/Mailchimp	Segmentación y automatizaciones
Contenido	Archivos Markdown	CMS headless con roles
Analítica	Vercel Analytics/PostHog	Eventos y embudos personalizados
Chat	Widget controlado	Agente conectado a conocimiento y handoff

16. Próxima fase: e-commerce

Agregar productos transforma alcance, riesgo y operación. Antes de elegir tecnología, define catálogo, variaciones, stock, monedas, impuestos, despacho, devoluciones, pagos y soporte.

Opción	Conviene cuando	Trade-off
Shopify	Quieres vender rápido con operación resuelta.	Costo mensual y personalización dentro de la plataforma.
WooCommerce	Ya usas WordPress y aceptas administrar plugins/hosting.	Mayor mantenimiento y superficie de seguridad.
Next.js + commerce	Necesitas experiencia muy personalizada e integración propia.	Más desarrollo, QA, seguridad y operación.
Checkout externo	Catálogo pequeño y venta simple.	Menos control, implementación rápida.

Prompt para decidir e-commerce

Analiza mi landing, productos, volumen esperado, país, moneda, despacho y equipo. Compara Shopify, WooCommerce y una solución Next.js. Evalúa costo total, tiempo, seguridad,

operación, SEO y capacidad de personalización. Recomienda una opción, explica descartes y propone hitos. No implementes todavía.

Prompt de arquitectura e-commerce

Diseña la arquitectura aprobada: catálogo, variantes, búsqueda, carrito, checkout, proveedor de pagos, verificación de webhooks, pedidos, inventario, emails, devoluciones, analítica y Product JSON-LD. Incluye threat model, ambientes, tests, rollback y checklist de lanzamiento.

17. Reto de siete días

Día	Foco	Evidencia
1	Publicar V1	URL y checklist básico
2	Claridad	Titular y CTA mejorados con feedback
3	Confianza	Proceso, caso o evidencia real
4	Móvil	Recorrido completo en 390 px
5	Contenido	FAQ y una página secundaria
6	Conversión	Formulario/CTA probado y evento medido
7	Calidad	QA, SEO y roadmap de la siguiente fase

CIERRE Publica, observa, corrige y repite. La evidencia de una versión real vale más que otra hora imaginando la versión perfecta.

Fuentes oficiales y documentación viva

Las interfaces, planes y versiones cambian. Verifica estos pasos en documentación oficial antes de una clase futura o una implementación sensible.

GitHub: crear repositorio: <https://docs.github.com/en/get-started/start-your-journey/creating-a-repository-for-your-project-on-github>

GitHub: ramas protegidas: <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches>

Vercel: deployments con Git: <https://vercel.com/docs/git>

Vercel: variables de entorno: <https://vercel.com/docs/environment-variables>

Next.js: metadata y OG: <https://nextjs.org/docs/app/getting-started/metadata-and-og-images>

Next.js: checklist de producción: <https://nextjs.org/docs/app/guides/production-checklist>

Astro: por qué Astro: <https://docs.astro.build/en/concepts/why-astro/>

Astro en Vercel: <https://docs.astro.build/en/guides/deploy/vercel/>

Google: crawling e indexación: <https://developers.google.com/search/docs/crawling-indexing>

Google: datos estructurados: <https://developers.google.com/search/docs/appearance/structured-data/intro-structured-data>

Google: robots.txt: <https://developers.google.com/search/docs/crawling-indexing/robots/intro>

OpenAI Docs: Codex y flujos: <https://learn.chatgpt.com/>